Analisi della Sintassi

1 Overview del programma

Il linguaggio Pike è un linguaggio la cui struttura sintattica è definita tramite la libreria `lark`.

1.1 Tipi

Il sistema di tipi di Pike è statico ma semplificato. I tipi supportati esplicitamente nella grammatica sono:

- **`num`**: Rappresenta valori numerici. Nel backend LLVM, questi sono trattati come `DoubleType` (virgola mobile a doppia precisione).
- **`str`**: Rappresenta stringhe di testo tra doppie virgolette. In LLVM sono trattate come puntatori a interi a 8 bit (`i8*`).
- **`void`**: Tipo di ritorno implicito per le funzioni che non restituiscono valori (non esiste una keyword esplicita `void` utilizzabile nei programmi).

I valori booleani non esistono come tipo distinto; vengono rappresentati come numeri (`double`), dove 0.0 è **falso** e 1.0 (o diverso da zero) è **vero**.

1.2 Commenti

Il linguaggio supporta commenti in stile shell (che iniziano con `#`), definiti tramite l'importazione `common.SH_COMMENT` nella grammatica Lark. Questi vengono ignorati dal parser.

2 Funzioni

2.1 Dichiarazione

Le funzioni vengono definite utilizzando la parola chiave `defunc`. La sintassi richiede un nome, una lista di parametri tra parentesi (opzionalmente vuota), un tipo di ritorno (opzionale) preceduto da `%`, e il body tra parentesi graffe.

La grammatica specifica che il tipo di ritorno esplicito può essere solo `% num`. Se omesso, la funzione è considerata `void`.

Esempio

```
# Funzione che accetta due numeri e ritorna un numero
defunc somma(a: num, b: num) % num {
    ret a + b;
}

# Funzione void (senza tipo di ritorno esplicito)
defunc saluta(nome: str) {
    !("Ciao ");
    !!(nome);
}
```

3 Statement

Gli statement rappresentano le azioni eseguibili nel corpo di una funzione o a livello globale. La maggior parte degli statement termina con un punto e virgola `;`, ad eccezione delle strutture di control flow come `if` e `for`.

3.1 Assegnazioni

L'assegnazione utilizza l'operatore `:=`. Se la variabile non esiste nella *symbol table* corrente, viene creata. Se esiste già, il suo valore viene aggiornato.

Esempio

```
x := 10;
y := x * 2;
```

3.2 Dichiarazione variabili

Non esiste una parola chiave specifica (come `var` o `let`) per la sola dichiarazione. La dichiarazione avviene implicitamente alla prima assegnazione (`:=`) o tramite la definizione dei parametri di una funzione.

3.3 Output

L'output a console è gestito tramite due operatori specifici che agiscono come wrapper per la funzione C `printf`:

- `!`: Stampa l'espressione senza andare a capo.
- `!!`: Stampa l'espressione aggiungendo un carattere di nuova riga (`\n`).

Esempio

```
!("Il risultato è: ");
!!(42);
# Output: Il risultato è: 42.000000
```

3.4 Return

L'istruzione `ret` termina l'esecuzione della funzione corrente. Può essere seguita da un'espressione (se la funzione ritorna un valore) o essere vuota (per funzioni `void`).

Esempio

```
ret 0; # Se la funzione ha % num
ret; # Se la funzione non ha un tipo di ritorno
```

3.5 If-Else

Il costrutto condizionale supporta un blocco `if` obbligatorio e un blocco `else` opzionale. La condizione è vera se il risultato dell'espressione è diverso da 0.0.

Esempio

```
if (x > 10) {
    !!("Maggiore");
} else {
    !!("Minore o uguale");
}
```

3.6 Loops

Il linguaggio supporta il ciclo `for` con una sintassi simile al C, ma con restrizioni specifiche nella grammatica. L'inizializzazione e l' incremento (step) devono essere numeri letterali (`NUMBER`), non espressioni generiche.

Esempio

```
# Ciclo da 0 a 9 con step 1
for (i, 0; i < 10; 1) {
    !!(i);
}
```

3.7 Espressioni

Le espressioni ritornano un valore e possono essere usate all'interno di statement o come argomenti.

3.7.1 Operazioni aritmetiche e di confronto

Le operazioni binarie seguono una precedenza standard definita nella grammatica (`comparison` < `sum` < `product` < `atom`).

- **Aritmetica:** `+` , `-` , `*` , `/` .
- **Confronto:** `<` (minore), `>` (maggiore). Il risultato è 1.0 (vero) o 0.0 (falso).
- **Atom:** Valori effettivi, come numeri, stringhe o il nome di una variabile.

3.7.2 Chiamate a funzione/procedura

Le chiamate a funzione avvengono scrivendo il nome seguito dagli argomenti tra parentesi.

Esempio

```
risultato := calcola(10, 20);
saluta("Mondo");
```